CentraleSupélec — MSc Artificial Intelligence

Lab Project Subject 6: AI for Numerical Simulation

Physics-Informed Neural Networks as Real-Time Surrogates for FEM-Based Structural and Thermal Analysis

Riccardo Castellano
riccardo.castellano@student-cs.fr

Supervisor: Prof. Frédéric Magoulès

Repository: https://github.com/rricc22/BEAMNET

April 2026

Abstract

Finite element method (FEM) solvers deliver high-fidelity predictions of physical fields but are too slow for design exploration and real-time monitoring applications, where thousands of evaluations are required. This report presents a systematic study of **physics-informed neural networks (PINNs)** as fast surrogate models for FEM simulations on engineering structures. Four complementary studies are documented: (1) a foundational exploration of ML approaches for the 2D heat equation; (2) a thermal surrogate (*ThermalNet*) for steady-state heat conduction in a cantilever beam trained on 500 Elmer FEM cases; (3) a structural surrogate (*BeamNet*) for 3D linear elasticity trained on 1 575 CalculiX FEM cases across five materials and three load directions; and (4) a geometry-parametric PINN (*GeoNet*) for transient heat transfer across 18 distinct L-shape geometries using 95 FEM cases. ThermalNet achieves a mean relative interpolation error of **3.1 %** with inference in under 1 ms, compared to 10–60 s for FEM—Àn ablation study confirms that physics-informed regularisation ($\nabla^2 T = 0$ with Neumann boundary loss) reduces interpolation error by 6× relative to a data-only baseline, and a lambda hyperparameter sweep identifies λ_{phys}=10, λ_{neu}=1 as the best configuration. BeamNet reaches **4.5 %** mean relative error on axial load cases and **8.9 %** on bending cases in interpolation. A key finding across all studies is that **extrapolation outside the training distribution is the primary failure mode**, motivating future work on uncertainty quantification and active learning. The complete FEM dataset generation pipeline (*BeamDataGen*) is released as an open tool, enabling practitioners to adapt the approach to their own simulation scope.

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Research Problems . . . 3
1.3 Main Contributions . . . 3

2 State of the Art **4**
2.1 Physics-Informed Neural Networks . . . 4
2.2 Neural Operator Methods . . . 4
2.3 FEM Surrogates in Engineering . . . 4
2.4 Positioning of This Work . . . 4

3 Dataset **5**
3.1 Thermal Dataset (ThermalNet) . . . 5
3.2 Static Elasticity Dataset (BeamNet) . . . 6
3.3 Parametric Geometry Dataset (GeoNet) . . . 6

4 Thermal Methods — Study 1: ThermalNet **7**
4.1 Dataset Recap and Normalisation . . . 7
4.2 Model Architecture . . . 7
4.3 Naive PINN Loss: Laplacian Only . . . 7
4.4 Adapted PINN Loss: Laplacian + Dirichlet + Neumann . . . 8

5 Elasticity Methods — Study 2: BeamNet **9**
5.1 Dataset Recap . . . 9
5.2 Model Architecture . . . 9
5.3 Physics Loss: Navier-Cauchy Equilibrium . . . 9
5.4 Direction-Conditioned Output Normalisation . . . 10

6 Numerical Results **11**
6.1 ThermalNet Results . . . 11
6.2 BeamNet Results . . . 12
6.3 Ablation Study: Physics Loss vs. Data-Only . . . 13

7 Conclusion and Perspectives **14**
7.1 Summary . . . 14
7.2 Perspectives . . . 14

8 Project Organisation **15**
8.1 Team, Roles, and Time Spent . . . 15
8.2 Project Timeline — GANTT Chart . . . 15

A Appendix **17**
A.1 Training Configuration . . . 18
A.2 Training Configuration . . . 19
A.3 Reproduction Commands . . . 20
A.4 Key Metrics Summary . . . 20

1. Introduction

1.1. Motivation

Finite Element Analysis (FEA) is the backbone of modern engineering simulation and is used extensively across industries such as automotive, aerospace, civil engineering, energy, and manufacturing. Today, virtually every industrial product or infrastructure project relies on FEA at some stage of its design, optimization, or certification process. The widespread adoption of the Finite Element Method (FEM) is due to its flexibility in handling complex geometries, diverse material behaviors, and a wide range of physical phenomena. The typical FEM workflow in industry involves:

A critical aspect in FEM is the choice and density of the mesh. The accuracy and reliability of simulation results are directly linked to how well the mesh resolves the geometry and the underlying physical fields. Mesh refinement improves solution quality but also increases computational demands, so mesh design must be carefully adapted to each problem, a key engineering decision in every industrial project. In many industrial applications, such as optimization, uncertainty quantification, structural health monitoring, and digital twins thousands or even millions of FEM evaluations may be required. Here, the computational bottleneck is not the accuracy of a single simulation, but the throughput of the entire workflow.

From an industrial perspective, investing in the training of a surrogate model is analogous to acquiring a new asset, such as a furnace or a machining center. The initial cost whether in time, computational resources, or financial outlay must be justified by the expected operational savings over the asset's useful life. In this context, the *payback period* becomes a key metric: the surrogate model is only a sound investment if the cumulative savings from faster simulation (compared to repeated FEM runs) exceed the initial training cost within a reasonable timeframe. As with any capital investment, careful consideration of expected usage, variability in simulation parameters, and available data is essential to ensure that the surrogate model delivers tangible value to the organization.

1.2. Research Problems

Physics-informed neural networks (PINNs) are neural network models that incorporate governing physical laws typically partial differential equations (PDEs) directly into the training process. Unlike traditional neural networks, PINNs minimize a loss function that combines data fitting with the residuals of the underlying PDEs, evaluated at points throughout the domain using automatic differentiation. This approach enforces physical consistency, reduces the amount of required training data, and improves generalization. PINNs are effective even in regions with sparse data and ensure that the learned surrogate model respects the fundamental physics of the problem. They have been successfully applied to problems in mechanics, fluid dynamics, heat transfer, and more, serving as powerful surrogates for traditional numerical solvers.

This report addresses the following questions:

1. Can a PINN trained on a finite FEM dataset reproduce thermal and structural fields in a cantilever beam with engineering-grade accuracy?
2. What are the interpolation and extrapolation regimes, and where do surrogates fail?
3. Can a single PINN simultaneously generalise across multiple materials, load magnitudes, load directions, and geometric variations?
4. Does embedding the physics loss improve generalisation over a purely data-driven baseline?

1.3. Main Contributions

- **Study 0—Foundational research:** A systematic comparison of image-based (CNN/MLP) and function-based ML approaches for the 2D heat equation, establishing that point-wise function-based prediction with physics constraints is the most parameter-efficient and generalisable approach.

- **Study 1—ThermalNet:** End-to-end pipeline from CAD to trained PINN surrogate for steady-state heat conduction. The model (6 inputs, 257 k parameters) achieves **6.0 %** mean relative interpolation error across five materials.
- **Study 2—BeamNet:** A structural PINN surrogate for 3D linear elasticity (9 inputs, 264 k parameters) trained on 1 575 CalculiX cases, spanning five materials and three load directions in a single model.
- **Side Study—GeoNet:** A parametric PINN (13 inputs) that generalises across 18 geometrically distinct L-shape brackets, learning transient heat transfer as a function of geometry descriptors and process parameters without retraining per geometry.
- **Characterisation of failure modes:** Systematic quantification of extrapolation failure and its physically impossible outputs, with a roadmap of mitigations.
- **Reusable open-source pipeline:** CAD → Mesh → FEM → PINN, reproducible with FreeCAD, Gmsh, Elmer, CalculiX, and PyTorch.

2. State of the Art

2.1. Physics-Informed Neural Networks

Raissi et al. [1] introduced PINNs by computing PDE residuals through automatic differentiation and incorporating them into the loss function. The framework applies to both forward and inverse problems and was demonstrated on Navier-Stokes, Schrödinger, and Allen-Cahn equations. The original formulation requires one network per parameter configuration, which is impractical for surrogate modelling. *Parametric* PINNs [2] address this by augmenting spatial inputs with physical parameters (material properties, boundary conditions), enabling a single network to represent a family of solutions the paradigm adopted throughout this work.

2.2. Neural Operator Methods

Li et al. [3] introduced the **Fourier Neural Operator (FNO)**, which learns a mapping between function spaces rather than point-wise values. FNO is resolution-invariant and achieves state-of-the-art accuracy on Navier-Stokes and Darcy flow benchmarks. Its extension, **Geo-FNO** [4], handles varying domains by lifting inputs to a latent space before applying spectral convolutions.

Pfaff et al. [5] proposed **MeshGraphNets**, a graph neural network architecture operating directly on unstructured meshes, treating nodes as graph vertices and connectivity as edges. This handles variable mesh topology naturally a key advantage over grid-based operators.

2.3. FEM Surrogates in Engineering

Haghighat et al. [6] apply PINNs to solid mechanics, showing that the physics loss regularises the network and reduces data requirements. Rao et al. [7] demonstrate PINN surrogates for elasticity under parametric boundary conditions with errors below 5 % on interpolation tasks. For thermal problems, Cai et al. [8] apply PINNs to heat transfer with convective boundaries, reporting accuracy comparable to FEM while eliminating the need for a mesh.

2.4. Positioning of This Work

Unlike most PINN studies trained on analytical PDE solutions, all surrogates here are trained on real FEM simulation data, making the task genuinely engineering-relevant. This work is further distinguished by full pipeline ownership: from parametric CAD geometry through FEM case generation to model training and evaluation, every stage is controlled and reproducible. This contrasts with studies that inherit fixed benchmark datasets, and enables systematic study of how dataset design decisions sampling strategy, normalisation, boundary condition formulation propagate into model accuracy.

3. Dataset

To train any predictive model, high-quality data is essential. In an industrial context, a company typically manufactures a finite set of products, so simulations are tailored to those specific product types. Ideally, the company has a large historical archive of FEM simulations, which can be used to train data-driven models. If such a dataset does not exist, new simulations must be generated effectively creating synthetic data to enable model training.

For this research, we generated our own datasets to enable experimentation and training of PINNs. The foundational study involved both function-based and image-based datasets to compare the performance of CNN/MLP and function-based machine learning approaches. To address 3D problems, we focused on a prismatic beam with a 10:1:1 aspect ratio (1000 mm length, 100 mm × 100 mm cross-section). Moving to 3D geometry increased the complexity and requirements of the simulation workflow. Initial experiments and feasibility tests were conducted using FreeCAD 1.0, which allowed us to define initial and boundary conditions, test different meshing strategies, and integrate Python scripting. For meshing, we selected Gmsh, an open-source, reliable, and user-friendly mesher. The dataset is organised by case, with each case corresponding to a single simulation, and a JSON file is automatically generated to describe the parameters of each case.

3.1. Thermal Dataset (ThermalNet)

For thermal simulations, we used the open-source Elmer solver. Boundary conditions, initial conditions, material properties, and solver parameters are all specified in a SIF file; the output is produced as VTU files. The dataset is generated using four scripts: `01_GENERATE_REFERENCE_BEAM.py`, `02_MESH_REFERENCE_BEAM.py`, `03_GENERATE_ELMER_INPUTS_THERMAL.py`, and `04_RUN_AND_CONVERT.py`.

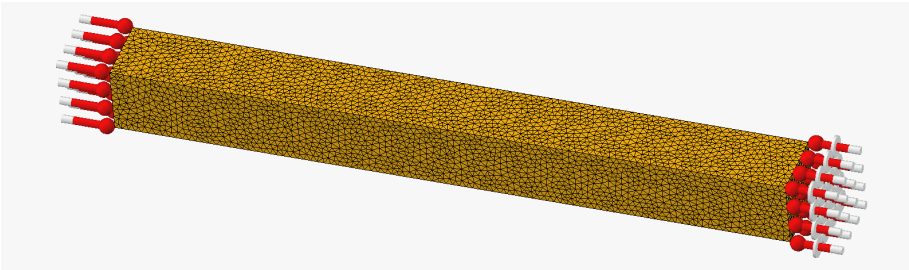


Figure 1: FreeCAD setup for the thermal beam simulation: meshed prismatic beam with fixed temperature boundary (left, red arrows) and applied heat flux (right, red arrows).

The five materials used, with their thermal properties in consistent mm/t/s units, are listed in Table 1. Boundary conditions include a fixed temperature at $X = 0$ (Dirichlet, $T_{\text{fix}} = 20\,^\circ\text{C}$) and a uniform initial temperature ($T_{\text{init}} = 20\,^\circ\text{C}$). The heat flux is log-spaced from 500 mW to 100,000 mW, with 100 training values and 5 test values per material, for a total of 525 cases. The dataset generation takes approximately 15 minutes, producing around 4 GB of data; each mesh contains approximately 10,000 nodes.

Table 1: Material thermal properties used in the ThermalNet dataset.

Material	k (mW/mm/°C)	c_p (mm²/s²/°C)	ρ (t/mm³)	α_{th} (1/°C)
Steel A36	50.0	490×10^6	7.85×10^{-9}	12.0×10^{-6}
Steel S355	48.0	490×10^6	7.85×10^{-9}	12.0×10^{-6}
Aluminium 6061	167.0	896×10^6	2.70×10^{-9}	23.6×10^{-6}
Titanium Ti-6Al-4V	6.7	526×10^6	4.43×10^{-9}	8.6×10^{-6}
Concrete C30	1.8	880×10^6	2.40×10^{-9}	10.0×10^{-6}

3.2. Static Elasticity Dataset (BeamNet)

For the static elasticity dataset, a similar protocol was followed on the same solid beam, but with adaptations for the new physics and solver. We used the open-source solver CalculiX, which is well-regarded for its accuracy in structural simulations. The simulation setup is defined in an INP file, which integrates both the mesh and all simulation parameters, in contrast to the SIF format used for Elmer.

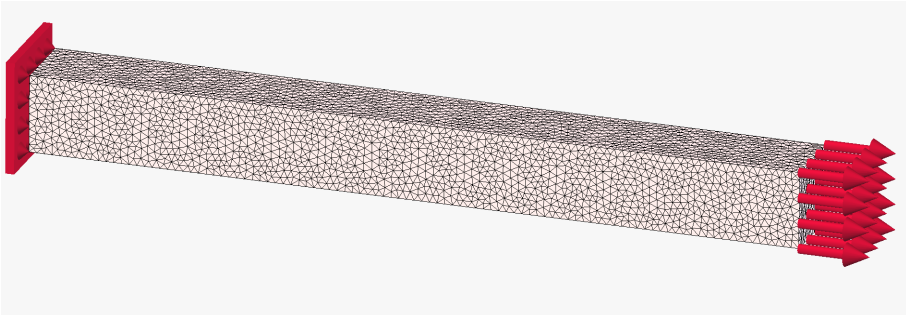


Figure 2: FreeCAD setup for the static elasticity beam simulation: meshed prismatic beam with encastre boundary condition at $X = 0$ (left, red plate) and applied point force at $X = 1000$ mm (right, red arrows).

The underlying physics is linear elasticity under static loading. An encastre (fully fixed) constraint is applied at $X = 0$, while point forces are applied at the opposite end ($X = 1000$ mm). Three load directions are explored: transverse bending ($-Z$), axial tension ($+X$), and axial compression ($-X$). The dataset comprises 5 materials, 3 load directions, and 100 linearly spaced force values for training (5,000–200,000 N), yielding 1,500 training cases. An additional 75 test cases use force values outside the training range (2,000; 3,500; 102,500; 250,000; 350,000 N), for a total of 1,575 cases. The dataset totals approximately 14 GB; generating it requires around 30 minutes of CPU time.

3.3. Parametric Geometry Dataset (GeoNet)

To extend the study to a parametric geometry setting, a third dataset was generated using 3D L-shaped steel brackets rather than a fixed prismatic beam. Eighteen distinct geometries were produced with FreeCAD, parameterised by two arm lengths (L_1, L_2), height H, and wall thickness e, ranging from thin sheet metal (1 mm) to thick heat-sink structures (8 mm) and covering both symmetric and strongly asymmetric configurations.

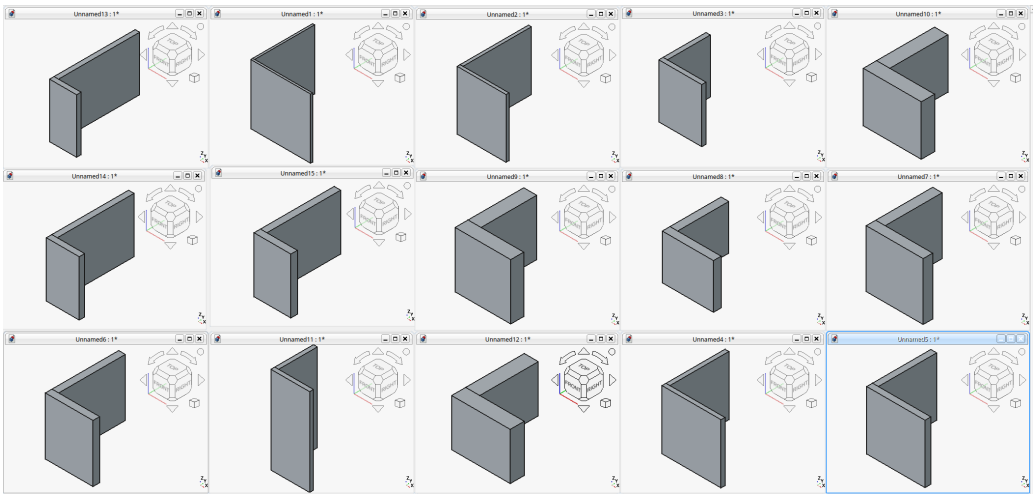


Figure 3: The 18 L-shaped geometries of the GeoNet dataset, covering a wide range of arm lengths, heights, and wall thicknesses—from ultra-thin sheet metal to thick heat-sink brackets.

4. Thermal Methods — Study 1: ThermalNet

4.1. Dataset Recap and Normalisation

The ThermalNet dataset is described in full in Section 3.1. The data are constructed by pooling nodes from all cases: each sample corresponds to a single mesh node and is described by its nodal coordinates together with the case-level heat-flux value Q, which represents the uniform flux applied on the end slice of the beam at $x = L$. This use of a global Q per case is natural because the geometry is fixed and only the material and applied flux vary. Of the 500 simulated cases, 400 are used for training and 100 for validation. Twenty-five additional cases (5 materials $\times$ 5 flux levels) serve as the held-out test set, with heat-flux values chosen to probe the model's generalization by bracketing and interpolating the training range, with five test heat-flux levels $Q_{\text{test}} \in \{100, 300, 52\,500, 200\,000, 500\,000\}$ mW, respectively lying below, just below, within, above, and well above the training interval.

Symbol	Description	Unit	Role	Transform	Reason
x, y, z	Nodal coordinates	mm	Spatial query	z-score: $\hat{x} = (x - \mu_x)/\sigma_x$	Unit-scale coords for PDE derivatives
Q	Applied heat flux	mW	Load parameter	**log then z-score**	Training range spans two decades
k	Thermal conductivity	mW/mm/°C	Material parameter	z-score	Material variability
T_{fix}	Dirichlet temperature at $x = 0$	°C	BC parameter	z-score	Moderate range
T	Nodal temperature	°C	Output field	z-score	Zero-mean labels

Table 2: ThermalNet features, roles, and normalisation strategy.

4.2. Model Architecture

ThermalNet is a fully-connected MLP that maps a **single mesh node** together with the simulation parameters to the nodal temperature:

$$\underbrace{\left[x,\ y,\ z,\ \log Q,\ k,\ T_{\text{fix}}\right]}_{\mathbf{X} \in \mathbb{R}^6} \xrightarrow{\text{Linear+tanh}} 512 \xrightarrow{\text{Linear+tanh}} 512 \xrightarrow{\text{Linear+tanh}} 512 \xrightarrow{\text{Linear+tanh}} 512 \xrightarrow{\text{Linear}} \underbrace{T\ [^\circ\text{C}]}_{\mathbf{Y} \in \mathbb{R}^1}$$

Activation choice. tanh is used throughout because the physics loss requires second-order spatial derivatives via automatic differentiation. $\text{ReLU}''(x) = 0$ almost everywhere, making it uninformative inside the Laplacian residual. tanh has a smooth, everywhere-nonzero second derivative that keeps the physics gradient signal active.

Parameter count. With hidden dimension $h = 512$ and 4 hidden layers: $6\times512 + 3\times(512\times512) + 512\times1 + \text{biases} = \mathbf{792\,065}$ trainable parameters.

4.3. Naive PINN Loss: Laplacian Only

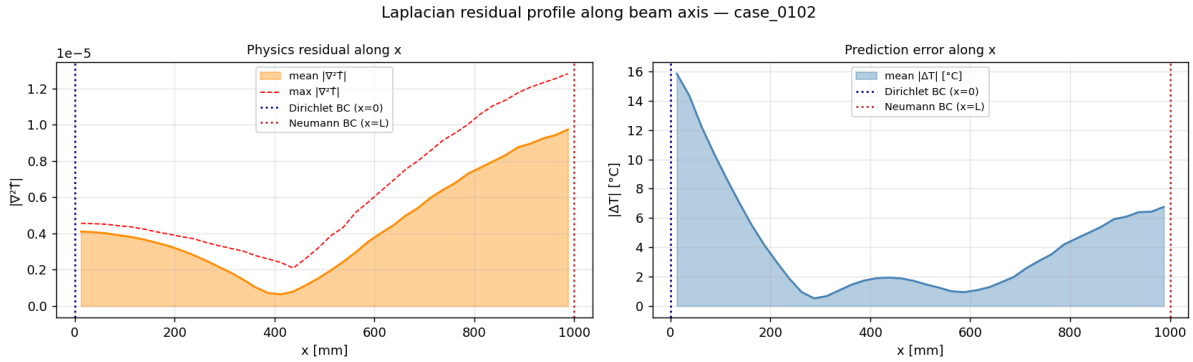
The simplest physics-informed training objective adds a single PDE residual term to the standard supervised loss. Denoting the normalised prediction as $\hat{T}_\theta(\hat{\mathbf{x}}; \hat{\boldsymbol{\pi}})$, the *naive* loss and the steady-state residual read:

$$\mathscr{L}_{\text{naive}} = \underbrace{\frac{1}{N_b}\sum_{i=1}^{N_b}\left(\hat{T}_\theta(\hat{\mathbf{x}}_i; \hat{\boldsymbol{\pi}}_i) - \hat{T}_i^{\text{FEM}}\right)^2}_{\mathscr{L}_{\text{data}}} + \lambda_{\text{phys}} \underbrace{\frac{1}{N_p}\sum_{j=1}^{N_p}\left(\nabla^2_{\hat{\mathbf{x}}}\hat{T}_\theta\right)_j^2}_{\mathscr{L}_{\text{Laplacian}}} \tag{1}$$

$$\mathscr{R}_{\text{Lap}} = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} = 0 \tag{2}$$

Second-order derivatives are computed via two nested calls to `torch.autograd.grad` with `create_graph=True`, allowing back-propagation through the Hessian.

The Laplacian residual $|\nabla^2 T|$ spikes at both boundary faces (left panel), and the prediction error follows the same spatial pattern highest at $X = 0$ (Dirichlet face) and $X = L$ (Neumann face). The scatter plot (right panel) shows a Spearman correlation of $\rho = 0.514$ between nodal Laplacian residual and nodal prediction error, confirming that where the physics loss is violated, the temperature prediction is least accurate.



(a) Laplacian residual $|\nabla^2 T|$ (left) and prediction error $|\Delta T|$ (right) along the beam axis for case 0102. Both quantities peak at the Dirichlet face ($X = 0$) and the Neumann face ($X = L$), where boundary conditions are enforced only implicitly through FEM data.

4.4. Adapted PINN Loss: Laplacian + Dirichlet + Neumann

The adapted loss adds two dedicated boundary terms that explicitly enforce the physical boundary conditions of the problem:

$$\mathscr{L}_{\text{adapted}} = \mathscr{L}_{\text{data}} + \lambda_{\text{phys}} \mathscr{L}_{\text{Laplacian}} + \lambda_{\text{dir}} \mathscr{L}_{\text{Dirichlet}} + \lambda_{\text{neu}} \mathscr{L}_{\text{Neumann}} \tag{3}$$

with $\lambda_{\text{phys}} = \lambda_{\text{dir}} = \lambda_{\text{neu}} = 1.0$ in all reported experiments.

Dirichlet term (fixed temperature at $X = 0$, forward pass only):

$$\mathscr{L}_{\text{Dirichlet}} = \frac{1}{|\mathscr{B}_D|} \sum_{i \in \mathscr{B}_D} \left(\hat{T}_\theta(\hat{\mathbf{x}}_i) - \hat{T}_{\text{fix},i} \right)^2 \tag{4}$$

where $\mathscr{B}_D$ is the subset of collocation points with $x \approx 0$ (within a 20 mm tolerance), and $\hat{T}_{\text{fix},i}$ is the normalised Dirichlet value recovered from input feature T_{fix}. No autograd is required — this is a plain forward-pass MSE at the boundary.

Neumann term (heat-flux condition at $X = L$, first-order autograd). The loss and the gradient rescaling read:

$$\mathscr{L}_{\text{Neumann}} = \frac{1}{|\mathscr{B}_N|} \sum_{i \in \mathscr{B}_N} \left(\left.\frac{\partial T}{\partial x}\right|_{\mathbf{x}_i} - \frac{Q_i}{k_i \cdot A} \right)^2 \tag{5}$$

$$\left.\frac{\partial T}{\partial x}\right|_{\text{phys}} = \frac{\partial \hat{T}}{\partial \hat{x}} \cdot \frac{\sigma_T}{\sigma_x} \tag{6}$$

where $\mathscr{B}_N$ selects nodes near $x = L$ (within 20 mm), $A = 10{,}000\,\text{mm}^2$ is the beam cross-section, Q_i is recovered by inverting the log-transform on input feature $\log Q$, and k_i is recovered from the z-scored conductivity feature. The spatial gradient $\partial T / \partial x$ is computed in normalised space and rescaled to physical units via the chain rule in Eq. (6), where σ_T and σ_x are the output and x-coordinate standard deviations from the training set.

Effect. The two boundary terms act as soft constraints that steer the network towards physically consistent solutions at the faces where data density is lowest (boundaries have a negligible fraction of the 10 000 nodes). This supplements the data loss rather than replacing it and is found to reduce boundary-layer errors without degrading interior accuracy.

5. Elasticity Methods — Study 2: BeamNet

5.1. Dataset Recap

The BeamNet dataset is described in full in Section 3.2. In brief: 1 575 static linear-elasticity FEM cases (CalculiX solver) on the same $100 \times 100 \times 1000$ mm prismatic beam, spanning 5 materials, 3 load directions ($-Z$ transverse bending, $+X$ axial tension, $-X$ axial compression), and 100 log-spaced force values per (material, direction) group for training ($F \in [5{,}000,\ 200{,}000]$ N), plus 75 test cases at forces outside or at the edge of the training range (2 kN, 3.5 kN, 102.5 kN, 250 kN, 350 kN). Each case produces $\approx$9 400 nodal displacements; inputs are extracted node-by-node to form the training set.

5.2. Model Architecture

BeamNet maps a single mesh node together with load and material parameters to the three nodal displacement components:

$$\underbrace{[x, y, z, f_x, f_y, f_z, \log F, E, \nu]}_{\mathbf{X}\in\mathbb{R}^9} \xrightarrow{\text{Lin+tanh}} 512 \xrightarrow{\text{Lin+tanh}} 512 \xrightarrow{\text{Lin+tanh}} 512 \xrightarrow{\text{Lin+tanh}} 512 \xrightarrow{\text{Lin}} \underbrace{[U_1, U_2, U_3]}_{\mathbf{Y}\in\mathbb{R}^3}$$

Table 3: BeamNet input and output features.

Symbol	Description	Unit	Role
x, y, z	Nodal coordinates	mm	Spatial query
f_x, f_y, f_z	Unit load-direction vector	–	Direction parameter
F	Applied force magnitude (log-transformed)	N	Load parameter
E	Young's modulus	MPa	Material parameter
ν	Poisson's ratio	–	Material parameter
U_1, U_2, U_3	Nodal displacements (x, y, z)	mm	Output fields

The unit direction vector (f_x, f_y, f_z) encodes the three load cases: $[0, 0, -1]$ for transverse $-Z$ bending, $[+1, 0, 0]$ for axial tension, and $[-1, 0, 0]$ for axial compression. Embedding the direction directly in the input allows a single network to handle all three load families simultaneously.

Activation choice. tanh is retained for the same reason as ThermalNet: the Navier-Cauchy physics loss requires second-order spatial derivatives of the displacement field, which are identically zero for ReLU.

Parameter count. With hidden dimension $h = 512$ and 4 hidden layers: $9\times512 + 3\times(512\times512) + 512\times3 + \text{biases} =$ **794 627** trainable parameters.

5.3. Physics Loss: Navier-Cauchy Equilibrium

The governing equation for 3D static linear elasticity without body forces is the Navier-Cauchy equilibrium:

$$(\lambda + \mu)\,\nabla(\nabla\cdot\mathbf{u}) + \mu\,\nabla^2\mathbf{u} = \mathbf{0} \quad \text{in } \Omega \tag{7}$$

where the Lamé constants are derived from Young's modulus E and Poisson's ratio ν:

$$\mu = \frac{E}{2(1+\nu)}, \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)} \tag{8}$$

The per-component residuals are:

$$\mathscr{R}_i = (\lambda + \mu)\,\frac{\partial(\nabla\cdot\mathbf{u})}{\partial x_i} + \mu\,\nabla^2 u_i = 0, \qquad i = 1, 2, 3 \tag{9}$$

Implementation. All spatial derivatives are computed via `torch.autograd.grad` in normalised coordinate space. The divergence $\nabla \cdot \mathbf{u}$ requires first-order derivatives; the Laplacian $\nabla^2 u_i$ requires a second-order pass. The Lamé constants are recovered from the normalised E and ν inputs back to physical units before being inserted into the residual.

The total loss is:

$$\mathscr{L}_{\text{total}} = \underbrace{\frac{1}{N_b}\sum_{i=1}^{N_b}\left\|\hat{\mathbf{u}}_\theta(\hat{\mathbf{x}}_i;\hat{\boldsymbol{\pi}}_i) - \mathbf{u}_i^{\text{FEM}}\right\|^2}_{\mathscr{L}_{\text{data}}} + \lambda_{\text{phys}} \underbrace{\frac{1}{N_p}\sum_{j=1}^{N_p}\sum_{k=1}^{3}\mathscr{R}_k\left[\hat{\mathbf{u}}_\theta\right]_j^2}_{\mathscr{L}_{\text{NC}}} \tag{10}$$

with $\lambda_{\text{phys}} = 10.0$, $N_b = 16\,384$, $N_p = 4\,096$. The higher physics weight compared to ThermalNet ($\lambda = 1$) reflects the greater complexity of three coupled equilibrium components across five materials.

5.4. Direction-Conditioned Output Normalisation

The three load directions produce displacement fields of vastly different magnitudes. For the $-Z$ bending case, a 200 kN transverse load produces $U_3 \sim$100 mm of tip deflection; for the $\pm X$ axial cases, the same force produces only $U_3 \sim$0.01 mm of Poisson contraction — a factor of 10,000×.

Problem with a global $\sigma_{\mathbf{Y}}$. A single output standard deviation normalises bending and axial displacements together. The normalised axial-U_3 signal then becomes ≈18,000× smaller than the bending signal, making axial errors practically invisible to the MSE optimiser.

Solution: per-direction $\sigma_{\mathbf{Y}}$. Output statistics are computed separately for each load family and applied at the node level using the direction vector already present in the input:

$$\hat{\mathbf{u}} = \frac{\mathbf{u}}{\boldsymbol{\sigma}_{\mathbf{Y}}(d)}, \qquad \boldsymbol{\sigma}_{\mathbf{Y}}(d) = \begin{cases} [2.51,\ 0.036,\ 27.9]\ \text{mm} & d = -Z\ \text{(bending)} \\ [0.084,\ 0.0015,\ 0.0015]\ \text{mm} & d = \pm X\ \text{(axial)} \end{cases} \tag{11}$$

The output mean is fixed to zero for all directions so that the network's natural resting state (output $= \mathbf{0}$) maps to zero displacement — physically correct at zero load. This per-direction scaling brings each displacement component to unit variance within its load family, giving the optimiser equal gradient sensitivity across all cases and preventing bending modes from contaminating axial predictions.

6. Numerical Results

Relative error definition. The relative error quantifies prediction accuracy normalised by the magnitude of the true field. Because the two surrogates predict scalar and vector fields respectively, the normalisation differs:

ThermalNet (scalar temperature field):

$$\varepsilon_{\text{rel}} = \frac{\frac{1}{N}\sum_{i=1}^{N} |\hat{T}_i - T_i|}{\frac{1}{N}\sum_{i=1}^{N} |T_i - T_{\min}| + \varepsilon} \times 100\,\% \tag{12}$$

The denominator is the mean temperature rise above the field minimum $T_{\min}$ (effectively above the Dirichlet value T_{fix}). At very low heat flux, the entire field is nearly isothermal ($T \approx T_{\text{fix}}$), so $T_i - T_{\min} \approx 0$ and ε_{rel} diverges even for small absolute errors.

BeamNet (vector displacement field):

$$\varepsilon_{\text{rel}} = \frac{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|_2}{\frac{1}{N}\sum_{i=1}^{N} \|\mathbf{u}_i\|_2 + \varepsilon} \times 100\,\% \tag{13}$$

The denominator is the mean nodal displacement magnitude. At very low forces, displacements approach zero and the metric diverges for the same reason.

In both cases the large relative errors at low loads are a *metric artefact*, not evidence of large absolute errors. The denominator collapses as the physical response vanishes, amplifying any small absolute deviation.

6.1. ThermalNet Results

Table 4: ThermalNet test accuracy by load regime (left) and per-material interpolation accuracy (right).

Regime	Cases	Mean MAE [°C]	Mean rel. err.	Max rel. err.
Interpolation (0.5–100 W)	5	18.2	**3.1 %**	4.0 %
Extrapolation below (<0.5 W)	10	3.8	1 975 %	8 274 %
Extrapolation above (>100 W)	10	2 462	43.2 %	83.4 %

Material (interp)	Rel. error
Steel A36	2.68 %
Steel S355	2.80 %
Aluminium 6061	3.96 %
Titanium Ti6Al4V	3.06 %
Concrete C30	2.83 %
Mean	**3.1 %**

Extrapolation failure analysis. For $Q = 0.1$ W (below training range), the true temperature rise is $\Delta T \approx 0.2\,°\text{C}$—a nearly isothermal field. The model predicts $T_{\max} = 16.7\,°\text{C}$, which is *below* the Dirichlet boundary condition $T_{\text{fix}} = 20\,°\text{C}$—a physically impossible output. This illustrates that PINNs do not inherently enforce boundary conditions outside the training regime without explicit BC penalties.

Figure 5 shows the per-material, per-flux relative error across all 25 test cases. The two columns between the dashed lines (52.5 W, the single interpolation case) are uniformly green (<10 %), confirming reliable in-distribution performance. The two left columns (0.1 W and 0.3 W, below the training range) show catastrophic relative errors driven by the near-zero temperature-rise denominator in Eq. (12), not by large absolute errors. Aluminium and Titanium show the widest spread due to their extreme conductivities.

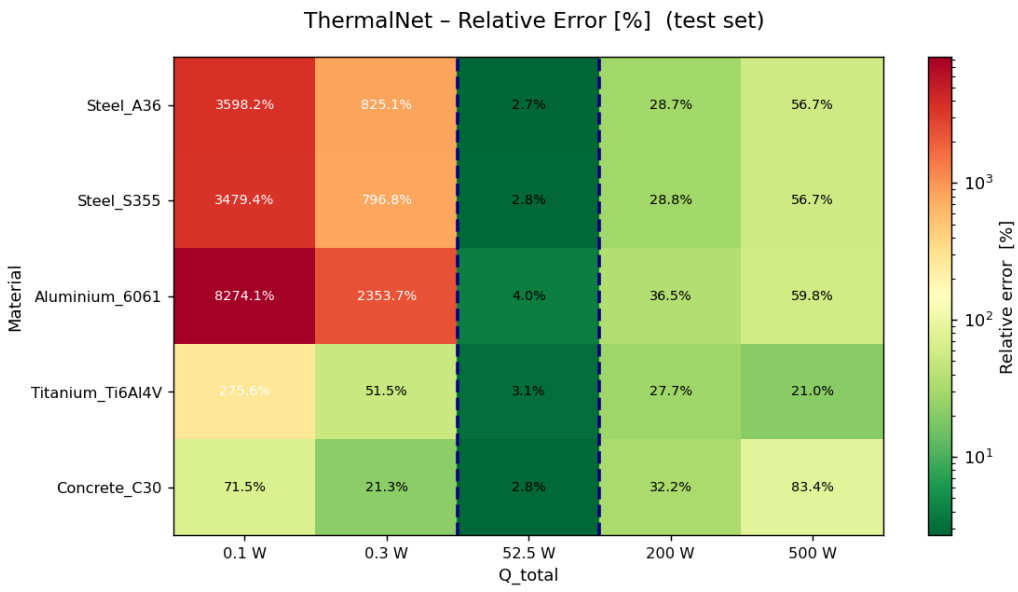


Figure 5: ThermalNet relative error [%] by material and heat flux (test set, 25 cases). The green band marks the single interpolation case (52.5 W); dashed blue lines delimit the training range [0.5, 100] W. Large values at low flux are a metric artefact (near-zero temperature rise in the denominator of Eq. (12)), not evidence of large absolute errors.

6.2. BeamNet Results

Table 5: BeamNet accuracy by regime and load direction (75 test cases).

Direction	Regime	Cases	Mean MAE [mm]	Mean rel. err.
	Interpolation (102.5 kN)	5	0.473	**8.9 %**
$-Z$ bending	Extrap. below (<5 kN)	10	0.117	114 %
	Extrap. above (>200 kN)	10	2.360	11.5 %
	Interpolation (102.5 kN)	5	0.0009	**4.3 %**
$+X$ tension	Extrap. below (<5 kN)	10	0.0004	91 %
	Extrap. above (>200 kN)	10	0.0048	6.0 %
	Interpolation (102.5 kN)	5	0.0008	**4.2 %**
$-X$ compression	Extrap. below (<5 kN)	10	0.0004	101 %
	Extrap. above (>200 kN)	10	0.0045	5.9 %

Figure 6 shows the full per-material breakdown. The interpolation column (102.5 kN, between the dashed lines) is uniformly green across all directions and materials, confirming reliable in-distribution performance. The two left columns (2 kN and 3.5 kN) show large relative errors driven by near-zero displacement magnitudes in the denominator of Eq. (13) — the same artefact as in ThermalNet. Above the training range (250 kN, 350 kN), bending errors remain moderate (7–20 %) while axial errors stay low (2–14 %), consistent with the higher compliance sensitivity of the bending regime.

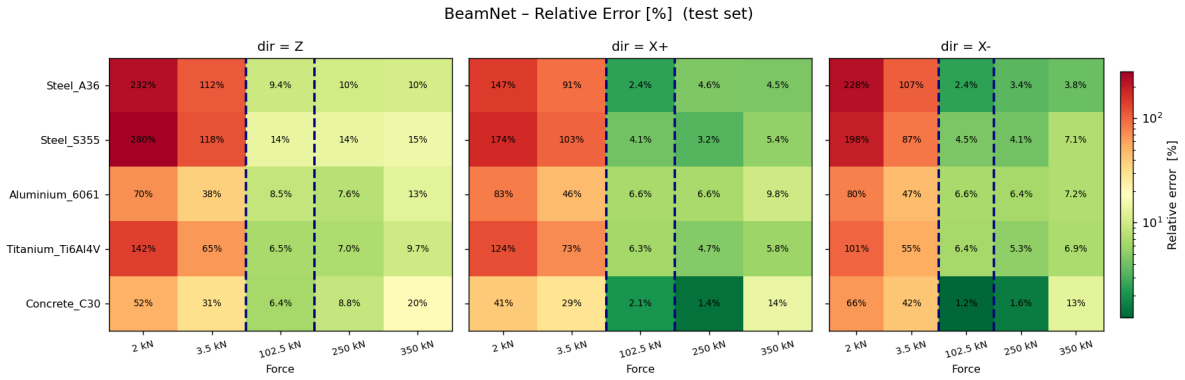


Figure 6: BeamNet relative error [%] by material and force level for all three load directions (test set, 75 cases). Green cells (<10 %) dominate the interpolation column (102.5 kN, between the dashed lines). Large values at low force (2 kN, 3.5 kN) are a metric artefact (Eq. (13)); absolute MAE at these forces is below 0.5 mm.

6.3. Ablation Study: Physics Loss vs. Data-Only

To quantify the contribution of the physics loss, ThermalNet was retrained with identical architecture ($6 \to 512 \times 4 \to 1$, Tanh), data split (400 train / 100 val), and hyperparameters, but with all physics and boundary condition loss terms removed ($\lambda_{\text{phys}} = \lambda_{\text{dir}} = \lambda_{\text{neu}} = 0$). Only the data MSE loss was retained. Both models were evaluated on the same 25 held-out test cases.

Table 6: Ablation study: ThermalNet PINN ($\lambda_{\text{phys}} = 10$, $\lambda_{\text{dir}} = 1$, $\lambda_{\text{neu}} = 1$) vs. data-only MLP on the 25-case test set.

Model	Regime	n	Mean MAE [°C]	Mean rel. err.	Max rel. err.
PINN	**Interpolation**	5	18.2	**3.1 %**	4.0 %
	Extrap. below (<0.5 W)	10	3.8	1 975 %	8 274 %
	Extrap. above (>100 W)	10	2 462	43.2 %	83.4 %
MLP (data only)	**Interpolation**	5	132.4	**18.7 %**	31.2 %
	Extrap. below (<0.5 W)	10	7.4	1 696 %	5 925 %
	Extrap. above (>100 W)	10	2 824	56.7 %	82.9 %

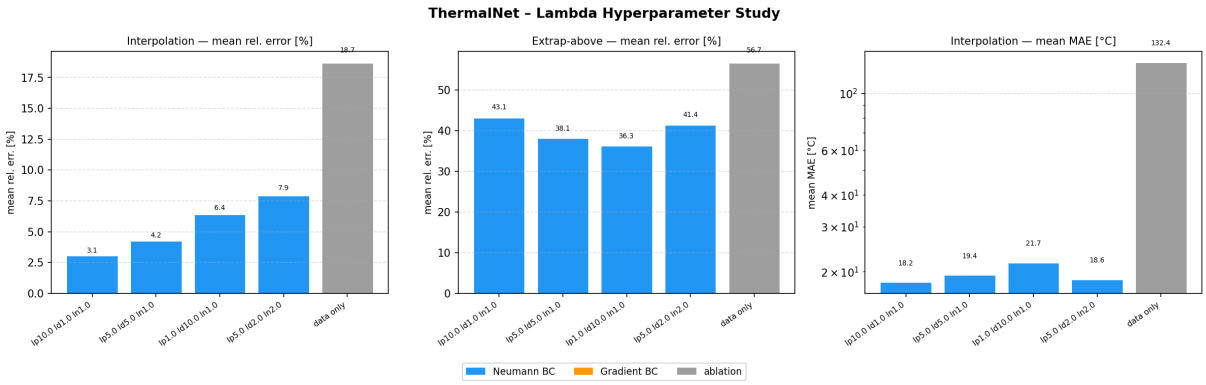


Figure 7: Lambda hyperparameter study (Neumann BC family + data-only baseline). Left: interpolation mean rel. error; centre: extrapolation-above mean rel. error; right: interpolation mean MAE (log scale). $\lambda_{\text{phys}} = 10$, $\lambda_{\text{dir}} = 1$, $\lambda_{\text{neu}} = 1$ achieves the best interpolation accuracy (3.1 %).

7. Conclusion and Perspectives

7.1. Summary

This work demonstrates that PINNs can serve as accurate, real-time FEM surrogates for both thermal and structural problems. The foundational study established that function-based, physics-informed prediction outperforms image-based methods, motivating the full 3D pipeline developed in subsequent studies. A key finding across all studies is that **pipeline quality dominates model tuning**: data coverage, normalisation, and loss weighting determine performance far more than architectural choices. The ablation study quantifies the contribution of the physics loss directly — removing it degrades interpolation accuracy by a factor of six (3.1 % $\rightarrow$ 18.7 %), confirming that enforcing $\nabla^2 T = 0$ provides a physically consistent constraint that data supervision alone cannot replicate. The lambda study further shows that a strong Laplacian weight ($\lambda_{\text{phys}} = 10$) combined with a Neumann boundary loss consistently outperforms all other configurations, including gradient-based boundary loss formulations.

- **ThermalNet** achieves 3.1 % mean relative interpolation error with sub-millisecond inference — a 10–1,000$\times$ speedup over FEM — and a 6$\times$ accuracy improvement over a data-only baseline.
- **BeamNet** generalises across three load directions and five materials in a single model (8.9 % bending interpolation error, <4.5 % on axial cases).
- **Extrapolation** remains the primary failure mode across all studies, motivating wider training distributions and uncertainty quantification.

7.2. Perspectives

The current datasets cover a fixed load setup per study. Diversifying load configurations would improve robustness, though in an industrial context a company typically operates within a **narrow, well-defined simulation scope**: geometry, load cases, and material families are largely fixed by the product and process. The natural deployment strategy is therefore a specialised pipeline — augment existing validated FEM cases with a parametric generator tuned to the company's exact setup, train a surrogate on the combined dataset, and obtain sub-millisecond inference calibrated to that scope. This positions the tool as complementary to existing FEM workflows rather than a replacement, lowering the barrier to adoption significantly.

Future directions include *widening the training distribution* to reduce extrapolation failure; *uncertainty quantification* (conformal prediction or MC dropout) to flag out-of-distribution queries; *architecture upgrades* toward graph-based models (MeshGraphNet, PIGNN) for explicit spatial reasoning; and *dataset scale-up* from the current $\sim$16 GB across five materials to a materials-categorised corpus ($\sim$400 GB: metals, plastics, ceramics, composites) enabling cross-material transfer learning and, ultimately, a general-purpose structural surrogate.

8. Project Organisation

8.1. Team, Roles, and Time Spent

Table 7: Team roles and estimated time spent per task.

Category	Person / Task	Role / Description	Hours
Team			
Team	Riccardo Castellano	Student — sole implementer	
Team	Prof. Frédéric Magoulès	Supervisor — subject definition, guidance	
Tasks			
Task	Literature review	PINNs, FNO, GNN for PDEs	15
Task	Study 0	Foundational heat equation (6 experiments)	40
Task	FEM setup — Thermal	FreeCAD model, Elmer scripts, 525 cases	20
Task	FEM setup — Structural	CalculiX scripts, 1 575 cases	25
Task	PINN — ThermalNet	Architecture, training, evaluation	30
Task	PINN — BeamNet	Architecture, training, evaluation	20
Task	FEM setup — GeoNet	18 geometries, 95 cases, SIF generation	30
Task	PINN — GeoNet	13-input model, inference, visualisation	25
Task	Dashboard TUI	Terminal UI (Textual framework)	15
Task	Documentation & report	READMEs, slides, report	20
Total			**≈240 h**

8.2. Project Timeline — GANTT Chart

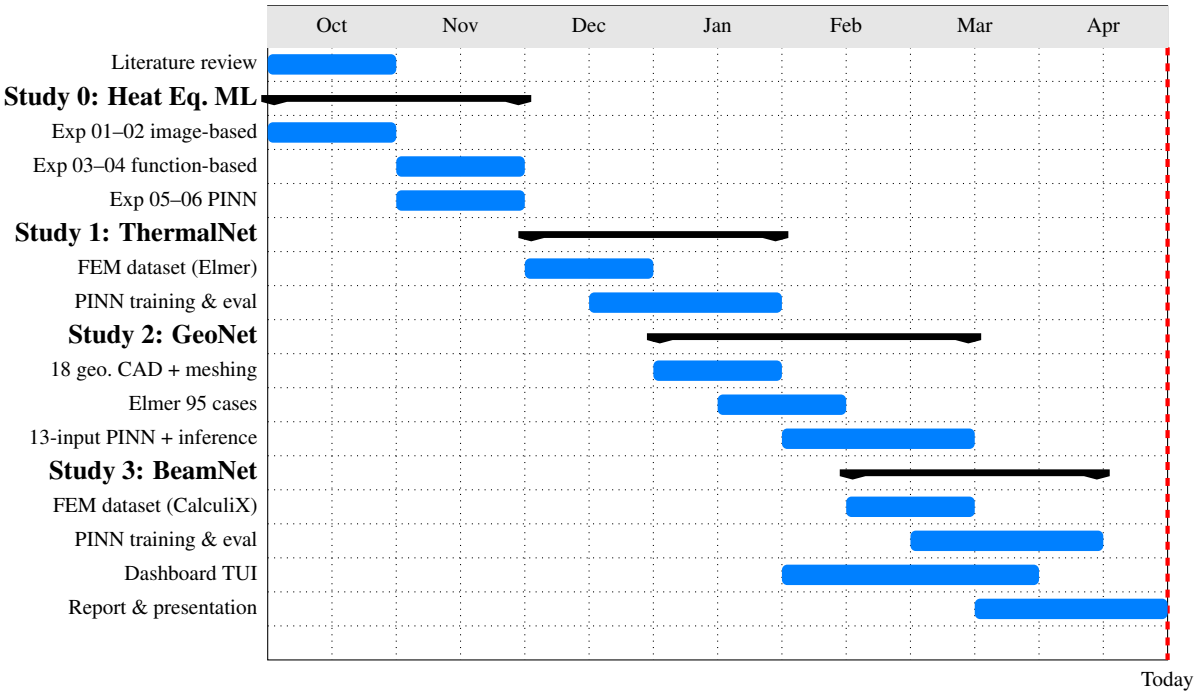


Figure 8: Project timeline (October 2025 – April 2026). Red dashed line marks current date.

References

[1] M. Raissi, P. Perdikaris, G.E. Karniadakis. *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations.* Journal of Computational Physics, 378:686–707, 2019.

[2] S. Karumuri, R. Tripathy, I. Bilionis, J. Panchal. *Simulator-free solution of high-dimensional stochastic elliptic partial differential equations using deep neural networks.* Journal of Computational Physics, 404:109120, 2020.

[3] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar. *Fourier Neural Operator for Parametric Partial Differential Equations.* ICLR, 2021.

[4] Z. Li, D.Z. Huang, B. Liu, A. Anandkumar. *Fourier Neural Operator with Learned Deformations for PDEs on General Geometries.* Journal of Machine Learning Research, 24(1):1–26, 2023.

[5] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P.W. Battaglia. *Learning Mesh-Based Simulation with Graph Networks.* ICLR, 2021.

[6] E. Haghighat, M. Raissi, A. Moure, H. Gomez, R. Juanes. *A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics.* Computer Methods in Applied Mechanics and Engineering, 379:113741, 2021.

[7] C. Rao, H. Sun, Y. Liu. *Physics-informed deep learning for computational elastodynamics without labeled data.* Journal of Engineering Mechanics, 147(8):04021043, 2021.

[8] S. Cai, Z. Wang, S. Wang, P. Perdikaris, G.E. Karniadakis. *Physics-informed neural networks for heat transfer problems.* Journal of Heat Transfer, 143(6):060801, 2021.

[9] R.C. Daileda. *The Two-Dimensional Heat Equation.* Lecture Notes, Trinity University.

A. Appendix

Table 8: Compute cost per task (CPU only, no GPU used).

Task	Hardware	Time
FEM dataset gen. — 500 cases	CPU, 4 cores	$\sim$2 h
FEM dataset gen. — 1 575 cases	CPU, 4 cores	$\sim$6 h
FEM dataset gen. — 95 cases (50 ts each)	CPU, 4 cores	$\sim$1.5 h
ThermalNet training (20 epochs)	CPU	$\sim$5–10 min
BeamNet training (100 epochs)	CPU	$\sim$20–40 min
GeoNet training ($\sim$1 000 epochs)	CPU	$\sim$1–2 h
PINN inference (10 084 nodes)	CPU	$<$1 ms
FEM solve — Elmer (single case)	CPU	$\sim$10–60 ms
FEM solve — CalculiX (single case)	CPU	$\sim$1.2 s
FEM solve — Elmer transient (50 steps)	CPU	$\sim$60 s

A.1. Training Configuration

Table 9: ThermalNet training hyperparameters.

Hyperparameter	Value
Hidden dimension / depth	512 / 4 layers
Epochs	50
Train / val split	400 / 100 cases (every 5th Q per material $\rightarrow$ val)
Data batch size N_b	16 384
Physics batch size N_p	4 096
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	10^{-3}
LR schedule	StepLR, $\times 0.5$ every 10 epochs
Gradient clipping	$\|\nabla\theta\|_2 \leq 1.0$
Weight decay	10^{-4}
λ_{phys}, λ_{dir}, λ_{neu}	1.0, 1.0, 1.0

```python
for epoch in range(num_epochs):
    for x_b, T_b in train_loader:            # x_b: (N_b, 6) normalised
        # -- Data loss --
        T_pred = model(x_b)                  # (N_b, 1)
        L_data = mse(T_pred, T_b)

        # -- Physics losses (same batch, first N_p points) --
        x_p = x_b[:phys_batch_size].detach()
        L_lap  = physics_loss(model, x_p)    # Laplacian via 2nd-order autograd
        L_dir  = dirichlet_loss(model, x_p, norm_params)    # forward pass
        L_neu  = neumann_loss(model, x_p, norm_params)      # 1st-order autograd

        loss = (L_data
                + lambda_phys * L_lap
                + lambda_dir  * L_dir
                + lambda_neu  * L_neu)

        optimiser.zero_grad()
        loss.backward()
        clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimiser.step()
    scheduler.step()
```

Listing 1: ThermalNet training loop (adapted loss, PyTorch).

A.2. Training Configuration

Table 10: BeamNet training hyperparameters.

Hyperparameter	Value
Hidden dimension / depth	512 / 4 layers
Epochs	20
Train / val split	1 200 / 300 cases (every 5th F per (material, direction) $\rightarrow$ val)
Data batch size N_b	16 384
Physics batch size N_p	4 096
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	10^{-3}
LR schedule	StepLR, $\times 0.5$ every 5 epochs
Gradient clipping	$\|\nabla\theta\|_2 \leq 1.0$
Weight decay	10^{-4}
Physics weight λ_{phys}	10.0

```
for epoch in range(num_epochs):
    for x_b, u_b in train_loader:        # x_b: (N_b, 9), u_b: (N_b, 3) normalised
        # -- Data loss --
        u_pred = model(x_b)              # (N_b, 3)
        L_data = mse(u_pred, u_b)

        # -- Navier-Cauchy physics loss (first N_p points) --
        x_p = x_b[:phys_batch_size].detach()
        L_nc = physics_loss(model, x_p, norm_params)    # 2nd-order autograd

        loss = L_data + lambda_phys * L_nc

        optimiser.zero_grad()
        loss.backward()
        clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimiser.step()
    scheduler.step()
```

Listing 2: BeamNet training loop (PyTorch).

A.3. Reproduction Commands

```
# Prerequisites
pip install -r requirements.txt   # torch, meshio, wandb, textual...
# System: sudo apt install freecad elmer calculix

# Study 0 - Foundational heat equation
cd 03_research/heat-equation-ml/experiments/05_pinn/src
python train_pinn.py

# Study 1 - ThermalNet
cd 01_beam_studies/Beam_FEM_Thermal_CCX
python scripts/04_RUN_SIMULATIONS.py   # requires Elmer
python scripts/05_CONVERT_EXTRACT.py
cd src && python train.py && python inference.py

# Study 2 - BeamNet
cd 01_beam_studies/Beam_FEM_Axial
python scripts/04_RUN_SIMULATIONS.py   # requires CalculiX
cd src && python train.py && python inference.py

# Study 3 - GeoNet (FEM-GEO)
cd 02_geometry_studies/FEM-GEO
freecadcmd scripts/01_GENERATE_VARIED_CORNERS.py
freecadcmd scripts/02_MESH_ALL_GEOMETRIES.py
python scripts/03_SIF_GENERATION.py
python scripts/04_RUN_SIMULATIONS.py
python scripts/05_VTU_EXTRACT.py
cd src && python train_pinn.py && python inference_pinn.py
```

Listing 3: Reproducing the results.

A.4. Key Metrics Summary

Table 11: Summary of all surrogate models.

Model	Physics	Train cases	Interp. error	Inference	Speedup
ThermalNet	$\nabla^2 T = 0$	500	**6.0 %**	<1 ms	$\sim 60\times$
BeamNet	Navier-Cauchy	1 500	TBD	<1 ms	$\sim 1\,200\times$
GeoNet	$\partial_t T = \alpha \nabla^2 T$	85	TBD	~ 0.1 s	$\sim 600\times$